PostgreSQL: Phân biệt Access Predicate và Filter Predicate

Phân biệt Access Predicate và Filter Predicate

Database PostgreSQL dùng ba method khác nhau để apply where clause (predicate):

Access Predicate (“Index Cond”)

Access predicate thể hiện start và stop condition của leaf node traversal.

Index Filter Predicate (“Index Cond”)

Index filter predicate chỉ được apply trong lúc leaf node traversal. Chúng không đóng góp vào start và stop condition, đồng thời không thu hẹp scanned range.

Table level filter predicate (“Filter”)

Predicate trên các column không thuộc index được evaluate ở table level. Để làm vậy, database trước hết phải load row từ heap table.

PostgreSQL execution plan không hiển thị riêng index access predicate và index filter predicate — cả hai đều xuất hiện dưới dạng “Index Cond”. Vì vậy, phải đối chiếu execution plan với index definition để phân biệt access predicate và index filter predicate.

> Note
>
> PostgreSQL explain plan không cung cấp đủ thông tin để tìm index filter predicate.

Các predicate được hiển thị dưới dạng “Filter” luôn là table level filter predicate — kể cả khi chúng được hiển thị cho một Index Scan operation.

Hãy xem ví dụ sau, vốn xuất hiện lần đầu trong chapter “Performance and Scalability”:

```
CREATE TABLE scale_data (
   section NUMERIC NOT NULL,
   id1     NUMERIC NOT NULL,
   id2     NUMERIC NOT NULL
);
CREATE INDEX scale_data_key ON scale_data(section, id1);
```

Select sau filter trên column ID2, column không được đưa vào index:

```
PREPARE stmt(int) AS SELECT count(*)
                       FROM scale_data
                      WHERE section = 1
                        AND id2 = $1;
EXPLAIN EXECUTE stmt(1);

                       QUERY PLAN
-------------------------------------------------------
Aggregate  (cost=529346.31..529346.32 rows=1 width=0)
  Output: count(*)
  -> Index Scan using scale_data_key on scale_data
     (cost=0.00..529338.83 rows=2989 width=0)
     Index Cond: (scale_data.section = 1::numeric)
     Filter: (scale_data.id2 = ($1)::numeric)
```

Predicate ID2 xuất hiện dưới dạng "Filter" bên dưới Index Scan operation. Điều này là vì PostgreSQL thực hiện table access như một phần của Index Scan operation. Nói cách khác, operation TABLE ACCESS BY INDEX ROWID của Oracle database bị ẩn bên trong PostgreSQL’s Index Scan operation. Vì vậy, một Index Scan có thể filter trên các column không nằm trong index.

> Important
>
> PostgreSQL Filter predicate là table level filter predicate — kể cả khi được hiển thị cho một Index Scan.

Khi thêm index từ chapter “Performance and Scalability”, ta thấy tất cả column đều xuất hiện dưới dạng “Index Cond” — bất kể chúng là access predicate hay filter predicate.

```
CREATE INDEX scale_slow
          ON scale_data (section, id1, id2);
```

Execution plan với index mới không hiển thị filter condition nào:

```
                        QUERY PLAN
-----------------------------------------------------
Aggregate  (cost=14215.98..14215.99 rows=1 width=0)
  Output: count(*)
  -> Index Scan using scale_slow on scale_data
     (cost=0.00..14208.51 rows=2989 width=0)
     Index Cond: (section = 1::numeric AND id2 = ($1)::numeric)
```

Lưu ý rằng condition trên ID2 không thể thu hẹp leaf node traversal vì index có column ID1 đứng trước ID2. Điều đó nghĩa là Index Scan sẽ scan toàn bộ range của condition SECTION=1::numeric và apply filter ID2=($1)::numeric trên từng row thỏa clause trên SECTION.

SQL Server

Method được mô tả trong section này áp dụng cho SQL Server Management Studio 2005 trở lên.

Getting an Execution Plan

Với SQL Server, có vài cách để lấy execution plan. Hai method quan trọng nhất là:

Graphically

Biểu diễn dạng graphical của SQL Server execution plan dễ truy cập trong Management Studio nhưng khó chia sẻ, vì thông tin predicate chỉ hiển thị khi di chuột qua operation tương ứng (“hover”).

Tabular

Execution plan dạng tabular khó đọc nhưng dễ copy vì hiển thị toàn bộ thông tin liên quan cùng lúc.

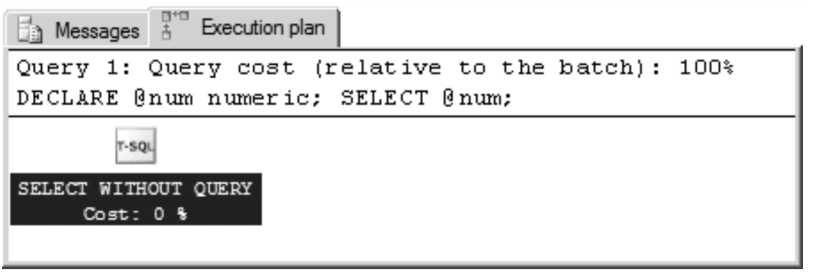
Graphically

Graphical explain plan được tạo bằng một trong hai button được highlight bên dưới.



Button bên trái explain statement đang được highlight trực tiếp. Button bên phải sẽ capture plan vào lần tiếp theo một SQL statement được execute.

Trong cả hai trường hợp, graphical representation của execution plan xuất hiện trong tab “Execution plan” của pane “Results”.



Graphical representation khá dễ đọc sau một chút thực hành. Tuy nhiên, nó chỉ hiển thị thông tin cơ bản nhất: các operation và table hoặc index mà chúng tác động lên.

Management Studio hiển thị thêm thông tin khi di chuột qua một operation (mouseover/hover). Điều này khiến việc chia sẻ execution plan với đầy đủ chi tiết trở nên khó khăn.

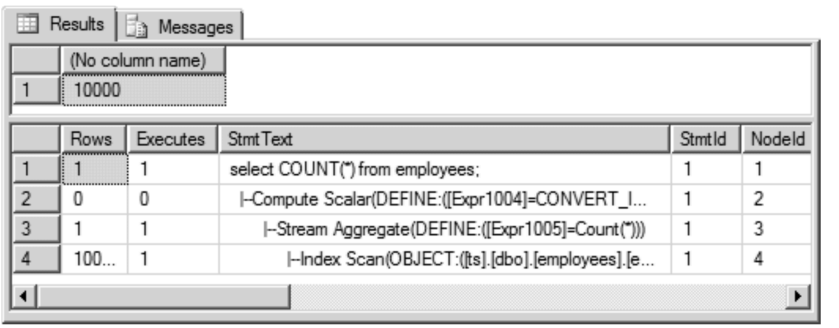
Tabular

Biểu diễn dạng tabular của SQL Server execution plan được lấy bằng cách profile việc execute một statement. Command sau bật chế độ đó:

```
SET STATISTICS PROFILE ON
```

Sau khi bật, mỗi statement được execute sẽ tạo thêm một result set. Chẳng hạn, select statement tạo ra hai result set — trước tiên là result của statement, sau đó là execution plan.

Execution plan dạng tabular hầu như không dùng được trong SQL Server Management Studio vì StmtText quá rộng để vừa màn hình.



Ưu điểm của cách biểu diễn này là có thể copy mà không làm mất thông tin quan trọng. Điều này rất tiện nếu bạn muốn đăng SQL Server execution plan lên forum hoặc nền tảng tương tự. Trong

trường hợp đó, thường chỉ cần copy column StmtText rồi format lại một chút:

```
select COUNT(*) from employees;
  |--Compute Scalar(DEFINE:([Expr1004]=CONVERT_IMPLICIT(...))
       |--Stream Aggregate(DEFINE:([Expr1005]=Count(*)))
            |--Index Scan(OBJECT:([employees].[employees_pk]))
```

Cuối cùng, bạn có thể tắt profiling lại:

```
SET STATISTICS PROFILE OFF
```

Operations

Index and Table Access

SQL Server có terminology đơn giản: operation “Scan” đọc toàn bộ index hoặc table, còn operation “Seek” dùng B-tree hoặc physical address (RID, giống Oracle ROWID) để truy cập một phần cụ thể của index hoặc table.

Index Seek

Index Seek thực hiện B-tree traversal và đi qua các leaf node để tìm mọi matching entry. Xem thêm “Anatomy of an Index” ở page 1.

Index Scan

Đọc toàn bộ index — tất cả row — theo index order. Tùy system statistics, database có thể thực hiện operation này nếu cần mọi row theo index order — chẳng hạn vì có order by clause tương ứng.

Key Lookup (Clustered)

Retrieve một row từ clustered index. Operation này tương tự Oracle INDEX UNIQUE SCAN cho một Index-Organized-Table (IOT). Xem thêm “Clustering Data” ở page 111.

RID Lookup (Heap)

Retrieve một row từ table — giống Oracle TABLE ACCESS BY INDEX ROWID. Xem thêm “Anatomy of an Index” ở page 1.

Table Scan

Còn được gọi là full table scan. Đọc toàn bộ table — tất cả row và column — theo cách chúng được lưu trên disk. Dù multi-block read operation có thể cải thiện đáng kể tốc độ Table Scan, đây vẫn là một trong những operation tốn kém nhất. Ngoài IO rate cao, Table Scan còn phải kiểm tra mọi table row nên có thể tiêu tốn nhiều CPU time. Xem thêm “Full Table Scan” ở page 13.

Join Operations

Nhìn chung, join operation chỉ xử lý hai table mỗi lần. Nếu query có nhiều join hơn, chúng được execute tuần tự: trước tiên là hai table, sau đó là intermediate result với table tiếp theo. Trong ngữ cảnh join, từ “table” vì vậy cũng có thể có nghĩa là “intermediate result”.

Nested Loops

Join hai table bằng cách lấy result từ một table rồi query table còn lại cho từng row của table thứ nhất. SQL Server cũng dùng nested loops operation để retrieve table data sau một index access. Xem thêm “Nested Loops” ở page 92.

Hash Match

Hash match join load các candidate record từ một phía của join vào hash table, sau đó probe hash table cho từng row từ phía còn lại của join. Xem thêm “Hash Join” ở page 101.

Merge Join

Merge join kết hợp hai sorted list như một zipper. Cả hai phía của join phải được presort. Xem thêm “Sort Merge” ở page 109.

Sorting and Grouping

Sort

Sort result theo order by clause. Operation này cần lượng memory lớn để materialize intermediate result (không pipelined). Xem thêm “Indexing Order By” ở page 130.

Sort (Top N Sort)

Sort một subset của result theo order by clause. Dùng cho top-N query nếu pipelined execution không thể thực hiện. Xem thêm “Querying Top-N Rows” ở page 143.

Stream Aggregate

Aggregate một presorted set theo group by clause. Operation này không buffer intermediate result — nó được execute theo pipelined manner. Xem thêm “Indexing Group By” ở page 139.

Hash Match (Aggregate)

Group result bằng hash table. Operation này cần lượng memory lớn để materialize intermediate result (không pipelined). Output không được order theo cách có ý nghĩa nào. Xem thêm “Indexing Group By” ở page 139.

Top-N Queries

Top

Abort các operation bên dưới khi đã fetch đủ số row mong muốn. Xem thêm “Querying Top-N Rows” ở page 143.

Hiệu quả của top-N query phụ thuộc vào execution mode của các operation bên dưới. Nó rất kém hiệu quả khi abort các non-pipelined operation như Sort.

Distinguishing Access and Filter-Predicates

Database SQL Server dùng ba method khác nhau để apply where clause (predicate):

Access Predicate (“Seek Predicates”)

Access predicate thể hiện start và stop condition của leaf node traversal.

Index Filter Predicate (“Predicates” hoặc “where” cho index operation)

Index filter predicate chỉ được apply trong lúc leaf node traversal. Chúng không đóng góp vào start và stop condition, đồng thời không thu hẹp scanned range.

Table level filter predicate (“where” cho table operation)

Predicate trên các column không thuộc index được evaluate ở table level. Để làm vậy, database trước hết phải load row từ heap table.

Section sau giải thích cách nhận diện filter predicate trong SQL Server execution plan. Nội dung dựa trên sample dùng để minh họa tác động của index filter predicate trong Chapter 3.

```
CREATE TABLE scale_data (
   section NUMERIC NOT NULL,
   id1     NUMERIC NOT NULL,
   id2     NUMERIC NOT NULL
);

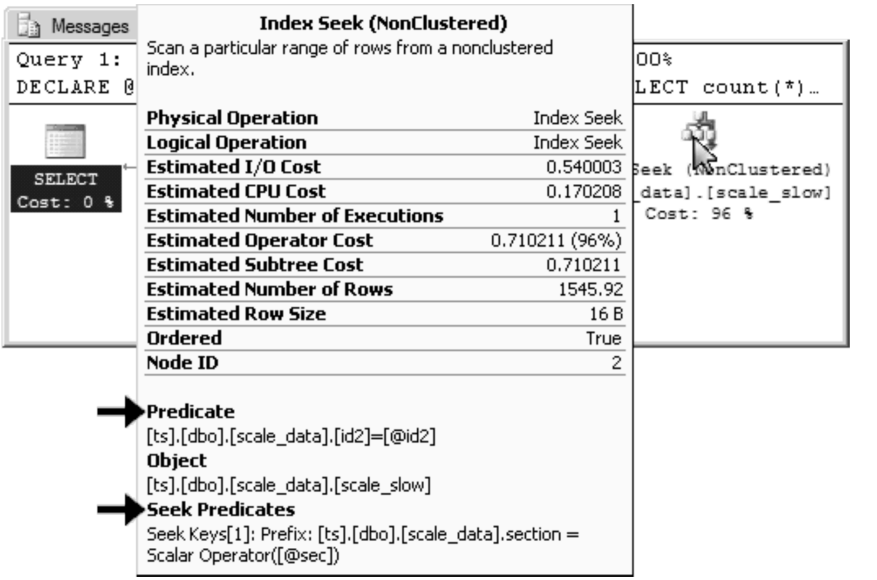
CREATE INDEX scale_slow ON scale_data(section, id1, id2);
```

Sample statement select theo SECTION và ID2:

```
SELECT count(*)
  FROM scale_data
 WHERE section = @sec
   AND id2 = @id2
```

In Graphical Execution Plans

Graphical execution plan ẩn thông tin predicate trong một tooltip chỉ xuất hiện khi di chuột qua Index Seek operation.



Seek Predicates của SQL Server tương ứng với access predicate của Oracle — chúng thu hẹp leaf node traversal. Filter predicate chỉ được gắn nhãn là Predicates trong graphical execution plan của SQL Server.

In Tabular Execution Plans

Tabular execution plan có thông tin predicate trong cùng column với nơi các operation xuất hiện. Vì vậy, rất dễ copy và paste toàn bộ thông tin liên quan trong một lần.

```
DECLARE @sec numeric;
DECLARE @id2 numeric;

SET STATISTICS PROFILE ON

SELECT count(*)
  FROM scale_data
 WHERE section = @sec
   AND id2 = @id2

SET STATISTICS PROFILE OFF
```

Execution plan xuất hiện dưới dạng result set thứ hai trong results pane. Sau đây là column StmtText — đã format lại một chút để dễ đọc:

```
|--Compute Scalar(DEFINE:([Expr1004]=CONVERT_IMPLICIT(...))
     |--Stream Aggregate(DEFINE:([Expr1008]=Count(*)))
          |--Index Seek(OBJECT:([scale_data].[scale_slow]),
             SEEK: ([scale_data].[section]=[@sec])
                   ORDERED FORWARD
             WHERE:([scale_data].[id2]=[@id2]))
```

Nhãn SEEK mở đầu cho access predicate, còn nhãn WHERE đánh dấu filter predicate.

MySQL

Method được mô tả trong section này áp dụng cho mọi version của MySQL.

Getting an Execution Plan

Đặt explain trước một SQL statement để lấy execution plan.

```
EXPLAIN SELECT 1;
```

Plan được hiển thị dưới dạng tabular (đã bỏ một số column ít quan trọng):

```
~+-------+------+---------------+------+~+------+------------~
~| table | type | possible_keys | key  |~| rows | Extra
~+-------+------+---------------+------+~+------+------------~
~| NULL  | NULL | NULL          | NULL |~| NULL | No tables...
~+-------+------+---------------+------+~+------+------------~
```

Thông tin quan trọng nhất nằm trong column TYPE. Dù tài liệu MySQL gọi đây là “join type”, tôi thích gọi nó là “access type” hơn vì thực tế nó chỉ cách data được truy cập. Ý nghĩa của từng type value được mô tả trong section tiếp theo.

Operations

Index and Table Access

Explain plan của MySQL dễ tạo cảm giác an toàn giả vì nói rất nhiều về việc index được sử dụng. Dù đúng về mặt kỹ thuật, điều đó không có nghĩa là index đang được dùng hiệu quả. Thông tin quan trọng nhất nằm trong column TYPE của MySQL explain output — nhưng ngay cả keyword INDEX ở đó cũng không cho biết việc indexing có đúng hay không.

eq_ref

Chỉ thực hiện B-tree traversal. Database dùng operation này khi primary key hoặc unique constraint bảo đảm search criteria match không quá một entry. Xem thêm “Anatomy of an Index” ở page 1.

ref, range

Thực hiện B-tree traversal và đi qua leaf node để tìm mọi matching entry (tương tự INDEX RANGE SCAN). Xem thêm “Anatomy of an Index” ở page 1.

index

Đọc toàn bộ index — tất cả row — theo index order (tương tự INDEX FULL SCAN).

ALL

Đọc toàn bộ table — tất cả row và column — theo cách chúng được lưu trên disk. Ngoài IO rate cao, table scan còn phải kiểm tra mọi row trong table nên có thể tạo tải CPU đáng kể. Xem thêm “Full Table Scan” ở page 13.

Using Index (trong column “Extra”)

Khi column “Extra” hiển thị “Using Index”, nghĩa là table không được access vì index có toàn bộ data cần thiết. Hãy nghĩ đến “using index ONLY”. Xem thêm “Clustering Data” ở page 111.

PRIMARY (trong column “key” hoặc “possible_keys”)

PRIMARY là tên của index được tự động tạo cho primary key.

Sorting and Grouping

using filesort (trong column “Extra”)

“using filesort” trong column Extra cho biết có explicit sort operation — bất kể sort diễn ra ở đâu (main memory hay disk). “Using filesort” cần lượng memory lớn để materialize intermediate result (không pipelined). Xem thêm “Indexing Order By” ở page 130.

Top-N Queries

implicit: không có “using filesort” trong column “Extra”

MySQL execution plan không hiển thị top-N query một cách rõ ràng. Nếu dùng limit syntax mà không thấy “using filesort” trong extra column, query được execute theo pipelined manner. Xem thêm “Querying Top-N Rows” ở page 143.

Distinguishing Access and Filter-Predicates

Database MySQL dùng ba cách khác nhau để evaluate where clause (predicate):

Access predicate (thông qua column “key_len”)

Access predicate thể hiện start và stop condition của leaf node traversal.

Index filter predicate (“Using index condition”, từ MySQL 5.6)

Index filter predicate chỉ được apply trong lúc leaf node traversal. Chúng không đóng góp vào start và stop condition, đồng thời không thu hẹp scanned range.

Table level filter predicate (“Using where” trong column “Extra”)

Predicate trên các column không thuộc index được evaluate ở table level. Để làm vậy, database trước hết phải load row từ table.

MySQL execution plan không cho biết condition nào dùng loại predicate nào — chúng chỉ liệt kê các loại predicate đang được sử dụng.

MySQL: Phân biệt Access và Filter Predicate

Trong ví dụ sau, toàn bộ where clause được dùng làm access predicate:

```
CREATE TABLE demo (
   id1 NUMERIC
 , id2 NUMERIC
 , id3 NUMERIC
 , val NUMERIC);

INSERT INTO demo VALUES (1,1,1,1);
INSERT INTO demo VALUES (2,2,2,2);

CREATE INDEX demo_idx
          ON demo
             (id1, id2, id3);

EXPLAIN
 SELECT *
   FROM demo
  WHERE id1=1
    AND id2=1;

+------+----------+---------+------+-------+
| type | key      | key_len | rows | Extra |
+------+----------+---------+------+-------+
| ref  | demo_idx | 12      |    1 |       |
+------+----------+---------+------+-------+
```

Trong column “Extra” không hiển thị “Using where” hoặc “Using index condition”. Tuy nhiên, index đã được sử dụng (type=ref, key=demo_idx), vì vậy có thể giả định toàn bộ where clause đủ điều kiện làm access predicate.

Có thể dùng key_len value để kiểm tra điều này. Nó cho biết query sử dụng 12 byte đầu tiên của index definition. Để map giá trị này với tên column, bạn “chỉ” cần biết mỗi column cần bao nhiêu storage space (xem “Data Type Storage Requirements” trong tài liệu MySQL). Khi không có NOT NULL constraint, MySQL cần thêm một byte cho

mỗi column. Sau cùng, mỗi NUMERIC column trong ví dụ cần 6 byte. Vì vậy, key length bằng 12 xác nhận hai index column đầu tiên được dùng làm access predicate.

Khi filter bằng column ID3 (thay vì ID2), MySQL 5.6 trở lên dùng index filter predicate (“Using index condition”):

```
EXPLAIN
 SELECT *
   FROM demo
  WHERE id1=1
    AND id3=1;

+------+----------+---------+------+-----------------------+
| type | key      | key_len | rows | Extra                 |
+------+----------+---------+------+-----------------------+
| ref  | demo_idx | 6       |    1 | Using index condition |
+------+----------+---------+------+-----------------------+
```

Trong trường hợp này, key length bằng six nghĩa là chỉ một column được dùng làm access predicate.

Các version MySQL trước đây dùng table level filter predicate cho query này — được nhận diện bởi “Using where” trong column “Extra”:

```
+------+----------+---------+------+-------------+
| type | key      | key_len | rows | Extra       |
+------+----------+---------+------+-------------+
| ref  | demo_idx | 6       |    1 | Using where |
+------+----------+---------+------+-------------+
```

Index

Symbols

2PC, *89*
?, :var, @var (see bind parameter)

A

Access Predicate, *44*
access predicates
  recognizing in execution plans
    Oracle, *170*
    PostgreSQL, *177*
    SQL Server, *185*
adaptive cursor sharing (Oracle), *75*
auto parameterization (SQL Server), *39*

B

B-tree (balanced search tree), *4*
between, *44*
bind parameter, *32*
  contraindications
    histograms, *34*
    `LIKE` filters, *47*
    partitions, *35*
  for execution plan caching, *32*
  type safety, *66*
bind peeking (Oracle), *75*
bitmap index, *50*
`Bitmap Index Scan` (PostgreSQL), *175*
Brewer's CAP theorem, *89*

C

CAP theorem, *89*
cardinality estimate, *27*
CBO (see optimizer, cost based)
clustered index, *122*
  transform to SQL Server heap table, *127*
clustering factor, *21*, *114*
  automatically optimized, *133*
clustering key, *123*
collation, *24*
commit
  deferrable constraints, *11*
  implicit for **`truncate table`**, *163*
  two phase, *89*
compiling, *18*
computed columns (SQL Server), *27*
constraint
  deferrable, *11*
  `NOT NULL`, *56*
cost value, *18*
`count(*)`
  often as index-only scan, *120*
  Oracle requires `NOT NULL` constraint, *57*
`COUNT STOPKEY`, *145*
cursor sharing (Oracle), *39*

D

data transport object (DTO), *105*
`DATE`
  efficiently working with, *62*
`DBMS_XPLAN`, *167*
`DEALLOCATE`, *174*
`DEFERRABLE` constraint, *11*
`DETERMINISTIC` (Oracle), *30*
`distinct`, *97*
`distinct()`
  in JPA and Hibernate, *97*
DML, *159*
doubly linked list, *2*
dynamic-update (Hibernate), *164*

E

eager fetching, *96*
eventual consistency, *89*
execution plan, *10*, *165*
  cache, *32*, *75*
  creating
    MySQL, *188*
    Oracle, *166*
    PostgreSQL, *172*
    SQL Server, *180*
  operations
    MySQL, *188*
    Oracle, *167*
    PostgreSQL, *174*
    SQL Server, *182*
`explain`
  MySQL, *188*
  Oracle, *166*
  PostgreSQL, *172*

F

FBI (see index, function-based)
`FETCH ALL PROPERTIES` (HQL), *105*
`fetch first`, *144*
filter predicates
  effects (chart), *81*
  recognizing in execution plans
    Oracle, *170*
    PostgreSQL, *177*
    SQL Server, *185*

full table scan, *13*
`All` (MySQL), *189*
`Seq Scan` (PostgreSQL), *174*
`TABLE ACCESS FULL` (Oracle), *168*
`Table Scan` (SQL Server), *183*
functions, *24*
deterministic, *29*
in partial indexes, *52*
window, *156*

G

`group by`, *139*
with PostrgesSQL and the Oracle database and an `ASC`/`DESC` index not pipelined, *140*

H

hash join, *101*
`HASH GROUP BY`, *169*
`HASH JOIN` (Oracle), *169*
`HASH Join` (PostgreSQL), *175*
`Hash Match`, *183*
`Hash Match (Aggregate)`, *184*
heap table, *3*, *122*
creating in SQL Server, *127*
Hibernate
eager fetching, *96*
`ILIKE` uses `LOWER`, *98*
updates all columns, *164*
hint, *19*

I

`IMMUTABLE` (PostgreSQL), *30*
index
covering, *117*
fulltext, *48*
function-based, *24*
case insensitive, *24*
to index mathematical calculations, *77*
join, *50*
limits
MySQL, Oracle, PostgreSQL, *121*
SQL Server, *122*
merge, *49*
multi-column, *12*
wrong order (effects), *81*
partial, *51*
prefix (MySQL), *121*
secondary, *123*
`index` in MySQL execution plans, *189*
index-only scan, *116*
index-organized table, *122*
database support, *127*
`Index Cond` (PostgreSQL), *177*
`INDEX FAST FULL SCAN`, *168*
`INDEX FULL SCAN`, *168*
`Index Only Scan` (PostgreSQL), *174*
`INDEX RANGE SCAN`, *167*
`Index Scan` (PostgreSQL), *174*
`Index Seek`, *182*
`INDEX UNIQUE SCAN`, *167*
when accessing an IOT, *124*
`INTERNAL_FUNCTION`, *67*
IOT (index-organized table), *122*

J

join, *91*
full outer, *109*

K

`Key Lookup (Clustered)`, *182*

L

lazy fetching
for scalar attributes (columns), *104*
leaf node, *2*
split, *160*
LIKE, *45*
alternatives, *48*
as index filter predicate, *112*
on DATE column, *67*
on `DATE` columns, *67*
`limit` (MySQL, PostgreSQL), *144*
logarithmic scalability, *7*
`LOWER`, *24*

M

`Merge Join`, *109*
PostgreSQL, *175*
SQL Server, *183*
`MERGE JOIN` (Oracle), *169*
multi-block read
for a full table scan, *13*
for a `INDEX FAST FULL SCAN`, *168*
MVCC, *163*
affects PostgreSQL index-only scan, *174*
myths
dynamic SQL is slow, *72*, *74*
most selective column first
disproof, *43*
origin, *49*
Oracle cannot index `NULL`, *56*

N

N+1 problem, *92*
`Nested Loops`, *92*
PostgreSQL, *175*
SQL Server, *183*
`NESTED LOOPS` (Oracle), *168*

NOSORT
SORT GROUP BY, *140*
WINDOW, *157*
NULL
indexing in Oracle, *54*

O

offset (MySQL, PostgreSQL), *148*
optimizer, *18*
cost based, *18*
hint, *19*
rule based, *18*
statistics, *21*
OPTIMIZE FOR (SQL Server), *76*
OPTION (SQL Server), *76*
OR
to disable filters, *72*
order by, *130*
ASC, DESC, *134*
NULLS FIRST/LAST, *137*
support matrix, *138*
OVER(), *156*

P

paging, *147*
offset method, *148*
seek method, *149*
approximated, *152*
parameter sniffing (SQL Server), *76*
parsing, *18*
partial index, *51*
partial objects (ORM), *104*
partitions and bind parameters, *35*
pipelining, *92*
PLAN_TABLE, *166*
predicate information, *20*
access vs. filter predicates, *44*
in execution plans
MySQL, *190*
Oracle, *170*
SQL Server, *185*
prepare (PostgreSQL), *172*
primary key w/o unique index, *11*

Q

query planner (see optimizer)

R

RBO (see optimizer, rule based)
RECOMPILE (SQL Server hint), *76*
result set transformer, *98*
RID, *3*
RID Lookup (Heap), *183*
root node, *5*
split, *160*
row sequencing, *113*
row values, *153*
ROWID, *3*
ROWNUM (Oracle pseudo column), *144*, *148*
ROW_NUMBER, *156*

S

scalability, *81*
horizontal, *87*
logarithmic, *7*
Scalability, *79*
Seek Predicates (SQL Server), *185*
select *, avoid to
enable index-only scans, *120*
improve hash join performance, *104*
Seq Scan, *174*
Sort (SQL Server), *184*
SORT GROUP BY, *169*
NOSORT, *140*
SORT ORDER BY, *130*
STOPKEY, *145*
SQL area, *75*
SQL injection, *32*
SSD (Solid State Disk), *90*
statistics, *21*
for Oracle function-based indexes, *28*
STATISTICS PROFILE, *181*
STOPKEY
COUNT, *145*
SORT ORDER BY, *146*
WINDOW, *157*
Stream Aggregate, *184*

T

top (SQL Server), *145*
Top-N Query, *143*
TO_CHAR(DATE), *66*
TRUNC(DATE), *62*
truncate table, *163*
triggers not executed, *163*

U

UPPER, *24*

V

Vaccum (PostgreSQL), *163*
virtual columns for NOT NULL constraints on FBI, *58*

W

where, *9*
conditional, *72*
in SQL Server execution plan, *187*
window functions, *156*

SQL Performance Explained! — Giờ thì sao?

Có thể bạn vẫn còn câu hỏi hoặc gặp một vấn đề rất cụ thể mà “SQL Performance Explained” chưa trả lời thỏa đáng? Instant Coaching là giải pháp cho bạn.

Instant Coaching

Instant Coaching là cách nhanh, dễ và không phiền phức để developer giải quyết các vấn đề khó về SQL database performance thông qua desktop sharing.

Instant Coaching không dùng các ví dụ dựng sẵn; các case thực tế được trình bày vì đó là cách tối ưu để hỗ trợ bạn giải quyết vấn đề hiện tại với SQL database chậm. Instant Coaching độc lập với vendor, hiệu quả… và có mức giá rất hợp lý!

Hãy thử ngay!

Đây là lúc tìm hiểu thêm về Instant Coaching! Hãy truy cập website của chúng tôi để biết thêm thông tin:

http://winand.at

SQL Performance Explained

SQL Performance Explained giúp developer cải thiện database performance. Trọng tâm là SQL — bao quát mọi SQL database chính mà không sa đà vào chi tiết của một product cụ thể nào.

Bắt đầu từ nền tảng về indexing và where clause, SQL Performance Explained dẫn developer đi qua mọi thành phần của một SQL statement và giải thích các vấn đề thường gặp của object-relational mapping (ORM) tool như Hibernate.

Các chủ đề được đề cập gồm:

» Sử dụng multi-column index

» Áp dụng SQL function đúng cách

» Sử dụng LIKE query hiệu quả

» Tối ưu join operation

» Clustering data để cải thiện performance

» Pipelined execution của order by và group by

» Đạt performance tốt nhất cho pagination query

» Hiểu khả năng scalability của database

Cấu trúc có hệ thống khiến SQL Performance Explained vừa là textbook vừa là reference manual mà mọi developer nên có trên bookshelf.

Bao quát

Oracle® Database · SQL Server® · MySQL · PostgreSQL

Về Markus Winand

Markus Winand đã phát triển SQL application từ năm 1998. Mối quan tâm chính của ông gồm performance, scalability, reliability và nói chung là mọi khía cạnh kỹ thuật khác của software quality. Hiện Markus làm trainer và coach độc lập tại Vienna, Austria.

http://winand.at/

ISBN 978-3-9503078-2-5



EUR 29.95
GBP 26.99

9 783950 307825 >